

LEAD University

Tarea 3: Polars

Pipeline de Datos y Machine Learning - Polars vs Pandas

Profesor: Johansell Villalobos Cubillo
Curso de Computación Paralela - 2026

1. Descripción del dataset

El conjunto de datos utilizado es Rossmann Store Sales, obtenido de la competencia del mismo nombre en Kaggle. Contiene el histórico de ventas diarias de 1.115 tiendas de la cadena Rossmann. El problema abordado es de regresión: predecir la variable Sales (ventas diarias de cada tienda).

Características del conjunto:

- Registros (train): 1.017.209 filas.
- Tiendas (store): 1.115, con metadatos usados para el join requerido.
- Tamaño del archivo: aproximadamente 38 MB.
- Variable objetivo: Sales (regresión).
- Variables numéricas: Customers, CompetitionDistance, Promo, Promo2, etc.
- Variables categóricas: StoreType, Assortment, StateHoliday, PromoInterval.

Valores faltantes: la columna CompetitionDistance presenta nulos (imputados con la mediana); las columnas de competencia y de Promo2 tienen ausentes (imputados con 0), y PromoInterval se completó con el valor "None". Decisiones de limpieza: se filtraron los registros con tienda cerrada (Open = 1) y ventas mayores a cero (Sales > 0), ya que solo interesan los días con actividad.

Nota de fuente: el dataset proviene de la competencia Rossmann Store Sales de Kaggle. Por restricciones de acceso a la API de Kaggle, se descargó desde un repositorio espejo público que contiene los archivos originales sin modificación.

2. Pipeline implementado

Se implementó el mismo pipeline en Polars y en Pandas para poder compararlos de forma justa. Las etapas son:

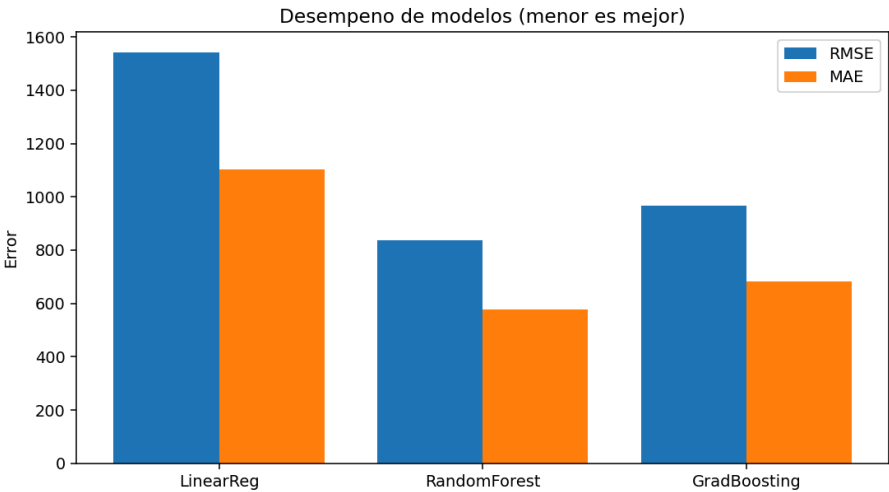
- Filtrado: se conservan registros con Open = 1 y Sales > 0.
- Join: se une train con store por la columna Store (left join).
- Manejo de faltantes: imputación con mediana, ceros y valor "None" según la columna.
- Transformación: extracción de Year, Month, Day y WeekOfYear de la fecha; codificación de StateHoliday, StoreType y Assortment a valores numéricos.
- Nuevas características: CompetitionOpenMonths (meses con competencia abierta) e IsPromoMonth (si el mes pertenece al intervalo de promoción).
- Agregación group_by: promedio de ventas y de clientes por tienda (AvgStoreSales, AvgStoreCustomers), reincorporados al conjunto mediante un join.

Nota técnica: las características AvgStoreSales y AvgStoreCustomers se calcularon sobre todo el conjunto. En un entrenamiento más riguroso deberían calcularse únicamente sobre el conjunto de entrenamiento para evitar fuga de información.

3. Resultados de Machine Learning

Se entrenaron tres modelos de regresión sobre las características generadas, usando una división 80/20 entre entrenamiento y prueba. Las métricas reportadas son RMSE y MAE (en ambas, un valor menor indica mejor desempeño).

Modelo	Tiempo entrenamiento (s)	RMSE	MAE
Regresión Lineal	0.18	1540.95	1102.87
Random Forest	18.50	835.83	577.57
Gradient Boosting	2.62	966.64	683.39



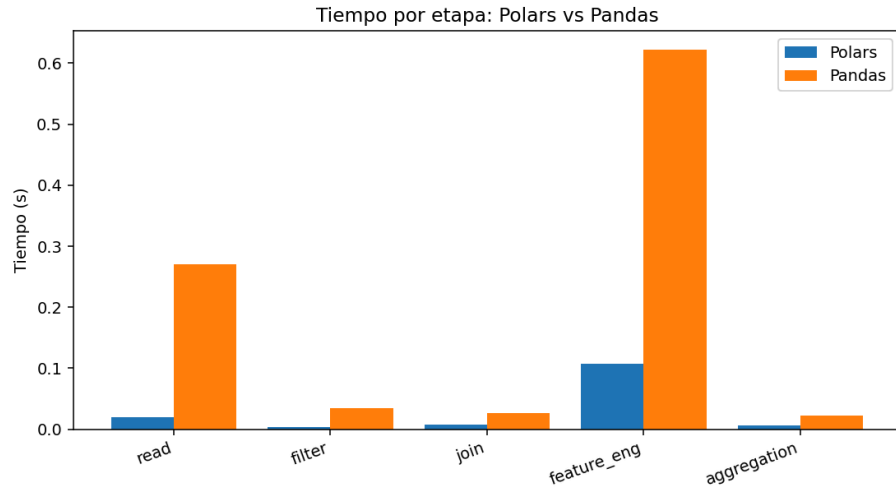
El Random Forest obtuvo el mejor desempeño predictivo (menor RMSE y MAE), aunque fue el más lento de entrenar. La Regresión Lineal fue la más rápida pero la menos precisa, lo que sugiere que la relación entre las variables y las ventas no es lineal.

4. Tablas comparativas Polars vs Pandas

Se midió el tiempo de cada etapa del pipeline en ambas bibliotecas. El speedup corresponde al cociente entre el tiempo de Pandas y el de Polars.

Etapas	Polars (s)	Pandas (s)	Speedup
Lectura (read)	0.0196	0.2698	13.74x
Filtrado (filter)	0.0031	0.0346	11.28x
Join	0.0081	0.0265	3.26x
Feature engineering	0.1072	0.6218	5.80x
Agregación	0.0058	0.0216	3.75x
Total	0.1437	0.9742	6.78x

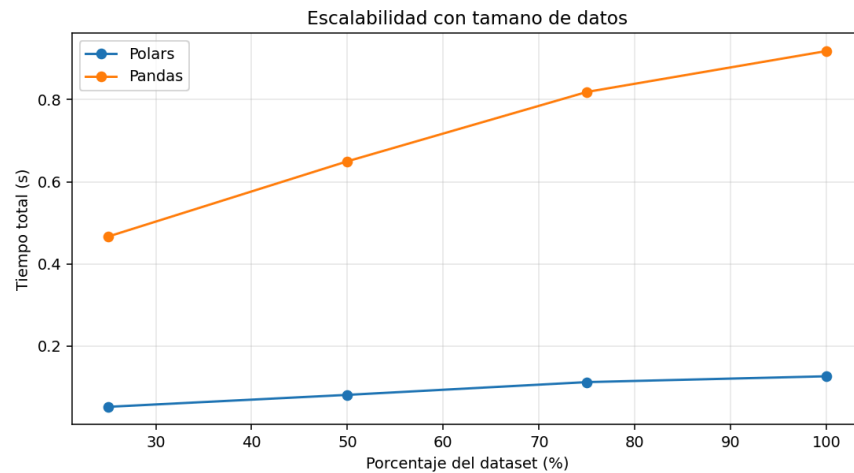
Información del sistema utilizado: 10 núcleos lógicos, 10 núcleos físicos, 17.18 GB de RAM, dataset de 1.017.209 filas (38.06 MB).

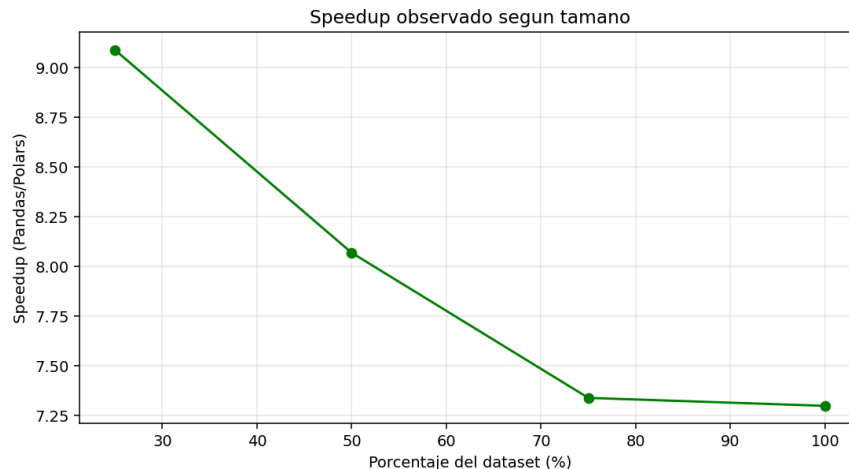


5. Experimento de escalabilidad

Se construyeron subconjuntos del 25%, 50%, 75% y 100% del dataset y se midió el tiempo total del pipeline en cada caso para ambas bibliotecas.

% del dataset	Polars (s)	Pandas (s)	Speedup
25%	0.0513	0.4661	9.09x
50%	0.0805	0.6498	8.07x
75%	0.1116	0.8189	7.34x
100%	0.1258	0.9186	7.30x





En todos los tamaños Polars resultó más rápido. El speedup disminuye levemente al crecer el dataset (de 9.09x a 7.30x), pero el tiempo de Pandas crece más rápido que el de Polars en términos absolutos.

6. Experimento de Lazy Execution

Se comparó la lectura y agregación usando `pl.read_csv` (modo eager) contra `pl.scan_csv().collect()` (modo lazy).

Métrica	Eager	Lazy
Tiempo de ejecución (s)	0.023	0.0299
Memoria (delta, MB)	-30.74	62.01

Los tiempos fueron similares porque la consulta evaluada es sencilla. El principal beneficio de la evaluación diferida aparece en pipelines más complejos, donde Polars optimiza el plan completo antes de ejecutarlo. La medición de memoria mediante el RSS del proceso es aproximada, por lo que debe tomarse como referencia y no como un valor exacto.

7. Discusión técnica y respuestas

1. ¿Qué ventajas observó al utilizar Polars?

Polars fue consistentemente más rápido que Pandas en todas las etapas, con un speedup total de 6.78x (0.14 s contra 0.97 s para procesar el millón de filas). Aprovecha los múltiples núcleos del sistema, usa expresiones vectorizadas y evaluación diferida, reduciendo tiempo y memoria.

2. ¿Qué operaciones obtuvieron el mayor speedup?

La lectura del CSV fue la de mayor aceleración (13.74x), seguida del filtrado (11.28x). Son operaciones donde Polars, apoyado en Apache Arrow y lectura paralela, supera ampliamente a Pandas.

3. ¿En cuáles operaciones la diferencia fue pequeña?

El join (3.26x) y la agregación con `group_by` (3.75x) tuvieron el menor margen. Siguen favoreciendo a Polars, pero son operaciones livianas en este dataset que Pandas resuelve relativamente rápido.

4. ¿Qué beneficios aporta Lazy Execution?

Permite construir un plan de ejecución completo que Polars optimiza antes de materializarlo con `collect()`, evitando cálculos intermedios innecesarios. En la prueba los tiempos eager y lazy fueron parecidos por ser una consulta simple; el beneficio crece en pipelines grandes y complejos.

5. ¿Qué limitaciones encontró en Polars?

La inferencia automática de tipos puede fallar: la columna `StateHoliday` mezcla "0" con letras a, b, c, y hubo que forzar su tipo con `schema_overrides`. Además su API difiere de Pandas, su ecosistema y documentación son menores, y para el modelado hay que convertir a Pandas/NumPy porque scikit-learn no trabaja directamente con Polars.

6. ¿Qué ventajas mantiene Pandas?

Ecosistema más maduro, integración directa con scikit-learn, matplotlib y la mayoría de librerías de ciencia de datos, más documentación y una API ampliamente conocida. Para datasets pequeños o flujos donde la velocidad no es crítica sigue siendo cómodo y suficiente.

7. ¿La aceleración observada justifica migrar un proyecto existente?

Depende del caso. Un speedup de 6.78x justifica migrar proyectos que procesan grandes volúmenes de datos de forma repetida, donde el tiempo de cómputo es un cuello de botella. Para proyectos pequeños o ya estables, el costo de reescribir y aprender la nueva API podría no compensar.

8. ¿Cómo afecta el tamaño del dataset al beneficio obtenido?

El speedup bajó levemente al crecer los datos: 9.09x al 25%, 8.07x al 50%, 7.34x al 75% y 7.30x al 100%. Polars siempre ganó, y el tiempo de Pandas creció más rápido que el de Polars.

9. ¿Qué modelo produjo el mejor desempeño predictivo?

El Random Forest, con el menor error (RMSE 835.83 y MAE 577.57), aunque fue el más lento de entrenar (18.50 s). El Gradient Boosting quedó segundo y la Regresión Lineal fue la más rápida pero la menos precisa, indicando que la relación no es lineal.

10. ¿Qué recomendaciones daría para proyectos futuros?

Usar Polars en la carga y el procesamiento de datos grandes, aprovechando lazy execution en pipelines complejos, y convertir a Pandas solo al final para el modelado. Definir los tipos de columnas explícitamente para evitar errores de inferencia. Para el modelado, priorizar modelos de ensamble como Random Forest o Gradient Boosting cuando la precisión importa, balanceando el mayor tiempo de entrenamiento.

8. Conclusiones

- Polars superó a Pandas en todas las etapas del pipeline, con un speedup global de 6.78x sobre un dataset de más de un millón de registros.
- Las mayores ganancias se dieron en la lectura del CSV y el filtrado; las menores en el join y la agregación.
- La ventaja de Polars se mantiene en todos los tamaños de datos probados, con una leve disminución relativa al aumentar el volumen.
- En el modelado, el Random Forest ofreció el mejor desempeño predictivo, a costa de un mayor tiempo de entrenamiento.
- Polars es una alternativa sólida para el procesamiento eficiente de datos grandes, complementándose con Pandas para la etapa de modelado.